

SKILL WEEK 5

1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char current[PATH_MAX];
    char previous[PATH_MAX] = "";
    char input[PATH_MAX];

    if (getcwd(current, sizeof(current)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    while (1)
    {
        printf("myshell:%s$ ", current);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "cd") == 0)
        {
            char *home = getenv("HOME");

            if (home == NULL)
            {
                printf("HOME not set\n");
                continue;
            }

            if (chdir(home) == -1)
            {
                perror("cd");
                continue;
            }
        }
        else if (strcmp(input, "cd -") == 0)
        {

```

```

        if (strlen(previous) == 0)
        {
            printf("Previous directory not available\n");
            continue;
        }

        if (chdir(previous) == -1)
        {
            perror("cd");
            continue;
        }
    }
else if (strncmp(input, "cd ", 3) == 0)
{
    char path[PATH_MAX];

    strcpy(path, input + 3);

    if (strlen(path) == 0)
    {
        printf("Path cannot be empty\n");
        continue;
    }

    if (chdir(path) == -1)
    {
        perror("cd");
        continue;
    }
}
else
{
    printf("Invalid command. Use cd <path>, cd -, cd or exit.\n");
    continue;
}

if (getcwd(current, sizeof(current)) == NULL)
{
    perror("getcwd");
    continue;
}

if (strlen(previous) == 0)
{
    strcpy(previous, current);
}
else
{
    char temp[PATH_MAX];

```

```

        strcpy(temp, previous);
        strcpy(previous, current);
        strcpy(current, temp);

        if (getcwd(current, sizeof(current)) == NULL)
        {
            perror("getcwd");
            continue;
        }
    }

    printf("Current directory: %s\n", current);
}

return 0;
}

```

OUPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill5.1.c -o skill5.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill5.1
myshell:/home/vishwa_teja_veerandi$ pwd
Invalid command. Use cd <path>, cd -, cd or exit.
myshell:/home/vishwa_teja_veerandi$ nano src/directory_navigation.c
Invalid command. Use cd <path>, cd -, cd or exit.
myshell:/home/vishwa_teja_veerandi$ cd
Current directory: /home/vishwa_teja_veerandi
myshell:/home/vishwa_teja_veerandi$ cd -
Current directory: /home/vishwa_teja_veerandi
myshell:/home/vishwa_teja_veerandi$ exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

2. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

typedef void (*command_func)(char *);

void builtin_cd(char *args)
{
    if (args == NULL || strlen(args) == 0)
    {
        char *home = getenv("HOME");

        if (home == NULL)
        {
            printf("HOME not set\n");
            return;
        }

        if (chdir(home) == -1)
            perror("cd");

        return;
    }

    if (chdir(args) == -1)
        perror("cd");
}

void builtin_pwd(char *args)
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("pwd");
        return;
    }

    printf("%s\n", cwd);
}

void builtin_export(char *args)
{
    if (args == NULL || strlen(args) == 0)
    {
        printf("Usage: export NAME=VALUE\n");
        return;
    }
}
```

```

    }

    char *equal = strchr(args, '=');

    if (equal == NULL)
    {
        printf("Invalid export syntax\n");
        return;
    }

    *equal = '\0';

    if (setenv(args, equal + 1, 1) == -1)
        perror("export");
}

void builtin_exit(char *args)
{
    exit(0);
}

struct command
{
    char *name;
    command_func function;
};

int main()
{
    struct command commands[] = {
        {"cd", builtin_cd},
        {"pwd", builtin_pwd},
        {"export", builtin_export},
        {"exit", builtin_exit}
    };

    int command_count = sizeof(commands) / sizeof(commands[0]);

    char input[1024];

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }

```

```

    if (strlen(input) == 0)
        continue;

    char *command = strtok(input, " ");
    char *args = strtok(NULL, "");

    int found = 0;

    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(command, commands[i].name) == 0)
        {
            commands[i].function(args);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Invalid command: %s\n", command);
}

return 0;
}

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill5.2.c -o skill5.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill5.2
myshell$ pwd
/home/vishwa_teja_veerandi
myshell$ cd/temp
Invalid command: cd/temp
myshell$ cd /temp
cd: No such file or directory
myshell$ hello
Invalid command: hello
myshell$ exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

3. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char cwd[PATH_MAX];
    char saved_directory[PATH_MAX];
    char input[100];

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(saved_directory, cwd);

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0)
        {
            if (getcwd(cwd, sizeof(cwd)) == NULL)
            {
                perror("getcwd");
                continue;
            }

            printf("%s\n", cwd);
        }
        else if (strcmp(input, "exit") == 0)
        {
            printf("Saving directory state: %s\n", cwd);
            strcpy(saved_directory, cwd);
            printf("Cleaning up resources...\n");
            printf("Shell terminated.\n");
            break;
        }
        else if (strcmp(input, "state") == 0)
```

```

    {
        printf("Saved directory: %s\n", saved_directory);
    }
    else
    {
        printf("Unknown command: %s\n", input);
    }
}

return 0;
}

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill5.3.c -o skill5.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill5.3
myshell$ hello
Unknown command: hello
myshell$ pwd
/home/vishwa_teja_veerandi
myshell$ exit
Saving directory state: /home/vishwa_teja_veerandi
Cleaning up resources...
Shell terminated.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

4. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

Create a file:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(const char *name)
{
    if (name == NULL || name[0] == '\\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

void handle_export(char *input)
{
    if (input == NULL || strlen(input) == 0)
    {
        printf("Usage: export NAME=VALUE\\n");
        return;
    }

    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("Invalid syntax. Use NAME=VALUE\\n");
        return;
    }

    *equal = '\\0';

    char *name = input;
    char *value = equal + 1;

    if (!valid_name(name))
    {
        printf("Invalid variable name: %s\\n", name);
        return;
    }
}
```

```

    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return;
    }

    printf("%s=%s\n", name, getenv(name));
}

int main()
{
    char input[1024];

    printf("Export Shell Started\n");

    while (1)
    {
        printf("export-shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strncmp(input, "export ", 7) == 0)
        {
            handle_export(input + 7);
        }
        else if (strncmp(input, "print ", 6) == 0)
        {
            char *name = input + 6;
            char *value = getenv(name);

            if (value != NULL)
                printf("%s=%s\n", name, value);
            else
                printf("%s is not set\n", name);
        }
        else
        {
            printf("Invalid command\n");
        }
    }
}

```

```

    printf("Export shell terminated.\n");

    return 0;
}

```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill5.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill5.4.c -o skill5.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill5.4
Export Shell Started
export-shell$ export NAME=Vishwa
NAME=Vishwa
export-shell$ print NAME
NAME=Vishwa
export-shell$ export NAME=students
NAME=students
export-shell$ print NAME
NAME=students
export-shell$ ^C
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```